

Rețele de Calculatoare, Proiect - Raport Tehnic - MonitorAndLogS

Timotei-Gruia Olaru

Facultatea de Informatică, Universitatea "Alexandru Ioan Cuza" din Iași, România
timotei-gruia.olaru@info.uaic.ro

Rezumat Acest document prezintă MonitorAndLogS, o aplicație client-server pentru monitorizarea resurselor sistemelor Linux. Soluția combină multiplexarea (`select`), pentru logarea periodică, cu procese dedicate (`fork`) pentru izolarea clienților, utilizând un protocol proprietar text pentru simplitate.

Keywords: Monitorizare · Client-Server · TCP/IP · Multiplexare I/O

1 Introducere

Proiectul **MonitorAndLogS** este o aplicație de tip client-server dezvoltată pentru monitorizarea eficientă a activității și a resurselor unui sistem Linux. Aplicația permite colectarea și stocarea de informații în fișiere log (format CSV) pentru a putea fi analizate ulterior. Serverul pune aceste date la dispoziția clienților, oferind statistici live sau istorice. Scopul principal este de a crea o unealtă simplă pentru supravegherea stării unui server pentru administratorii de sistem.

2 Tehnologii Aplicate

Pentru implementarea proiectului au fost utilizate următoarele tehnologii și concepte:

- **Protocolul TCP/IP:** Protocolul de transport utilizat pentru a asigura o comunicare fiabilă între client și server. Am optat pentru TCP deoarece livrarea ordonată și corectitudinea datelor transferate sunt foarte importante.
- **Multiplexare I/O:** Prin primitiva `select()`: Este utilizat în procesul părinte pentru a gestiona socket-ul de ascultare și mecanismul de timer fără a bloca așteptarea noilor conexiuni.
- **Fork:** Gestionarea sesiunii unui client se face prin crearea unui proces copil dedicat. Astfel se asigură separarea clienților și previne blocarea buclei principale de evenimente.
- **Memorie Partajată (POSIX Shared Memory):** Pentru a permite proceselor copil să modifice intervalul de logare al procesului părinte, se folosește un segment de memorie partajată (`shm_open/mmap`).
- **Sistemul de fișiere Linux (/proc) + comenzi:** Colectarea datelor se face prin interogarea directă a kernel-ului (`/proc/loadavg`, `/proc/meminfo`) sau prin parsarea output-ului unor comenzi precum `ss`, `who` și `ps`.
- **SFML:** Utilizată pentru dezvoltarea interfeței grafice (GUI) a clientului. Aceasta simplifică redarea ferestrelor, a textului și a primitivelor grafice (linii, dreptunghiuri) necesare pentru graficele de monitorizare în timp real.

- //Acest proiect folosește biblioteca SFML, aflată sub licența zlib/png, care este compatibilă integral cu GNU GPL (poate fi folosită ca dependentă, iar proiectul rămâne în continuare sub GPL, este necesară includerea licenței doar dacă se aduc modificări codului sursă al bibliotecii).
- //Din câte am înțeles de la domnul profesor de laborator, este permisă utilizarea componentelor aflate sub licențe compatibile GPL, cum ar fi MIT.
- //Dacă totuși utilizarea bibliotecilor externe nu este permisă, am inclus și varianta de client CLI ca alternativă de rezervă.

3 Structura Aplicației

Aplicația urmează o arhitectură clasică Client-Server cu un server concurent orientat pe procese.

- **Serverul:** Rulează ca un serviciu de fundal și folosește o buclă infinită de evenimente. Acesta monitorizează descriptorii de fișier pentru noi conexiuni + scurgerea timpului (pentru logarea periodica). La stabilirea unei conexiuni, serverul creează un proces copil care moștenește conexiunea și gestionează exclusiv dialogul cu clientul, logica de autentificare și interpretarea comenzilor. Este componenta centrală care rulează pe sistemul monitorizat.
- **Clientul CLI:** Este o aplicație interactivă în consolă care permite utilizatorului să interacționeze cu serverul folosind comenzi text (este inclus și separat, poate fi folosit și în secțiunea Consola a clientului GUI).
- **Clientul GUI:** O alternativă grafică ce oferă o experiență vizuală superioară.
 - **Arhitectură Single-Threaded:** Logica de rețea este integrată direct în bucla principală de evenimente SFML, utilizând un timer neblocaant (`sf::Clock`) pentru actualizarea datelor la fiecare secundă. Aceasta elimină complexitatea sincronizării (mutex) și asigură o interactivitate fluidă.
 - **Interfață Modulară:** Aplicația este structurată pe "scene" (Login, Dashboard, Procese, Rețea, etc.), fiecare având funcții dedicate pentru desenare și procesare de evenimente.
 - **Funcționalități Vizuale:** Include grafice liniare cu gradient pentru istoricul resurselor (Load, CPU, RAM, Nr. Procese) și tabele colorate sintactice pentru procese și conexiuni.

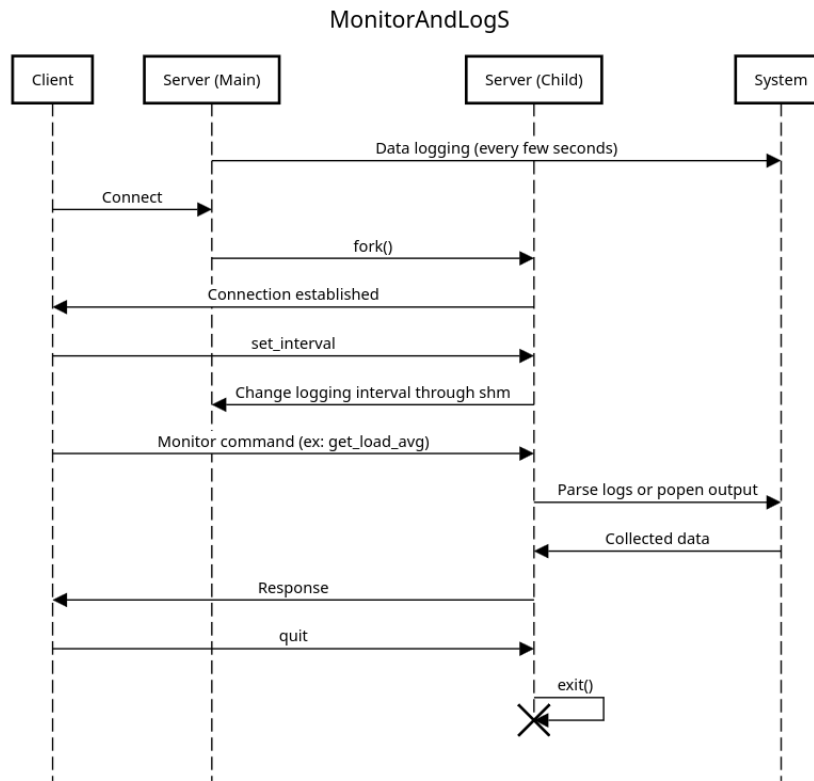


Figura 1. Diagrama Aplicației

4 Protocolul de Comunicare

Protocolul de comunicare, la nivelul aplicație, este unul proprietar, orientat pe text. Acesta urmează modelul cerere/raspuns (request/reply), mesajele fiind codificate în format ASCII. Fiecare schimb de mesaje constă într-o cerere inițiată de client și un răspuns furnizat de server.

Specificații Tehnice:

- **Formatul Mesajelor:** Clientul trimite comenzi sub formă de șiruri de caractere. Separatorul dintre comandă și argumente (dacă există) este spațiul. Lungimea maximă a unui mesaj este limitată de buffer-ul definit în aplicație (16384 octeți).
- **Gestionarea Stării:** Protocolul menține o stare a sesiunii. Serverul reține un flag (auth) pentru fiecare conexiune activă. Comenzile de monitorizare sunt respinse cu un mesaj de eroare dacă clientul nu s-a logat cu succes mai întâi.

- **Mecanismul de Răspuns:** Serverul procesează cererea și returnează un singur mesaj de răspuns, care poate fi o confirmare simplă, un set de date formatat sau un mesaj de eroare.

Comenzi definite:

1. **login <user> <parola>:** Inițiază sesiunea. Validează credențialele din fișierul `users.txt` și schimbă starea internă a conexiunii în modul autentificat.
2. **logout:** Invalidarea sesiunii curente, fără a închide conexiunea TCP.
3. **help:** Afișează lista de comenzi disponibile.
4. **stats:** Trimite toate comenzile de monitorizare și afișează toate statisticile aferente.
5. **quit:** Solicită închiderea conexiunii. Serverul închide socket-ul asociat acestui client.
6. **get_load_avg:** Afișează media încărcării sistemului (1 min, 5 min, 15 min).
7. **get_proc_info [n]:** Afișează un rezumat al proceselor active. Argumentul opțional `n` specifică numărul de top procese consumatoare de CPU care să fie afișate.
8. **get_logged_users:** Afișează lista utilizatorilor conectați la sistem.
9. **get_services:** Afișează lista serviciilor (porturile deschise în modul listen) pe care serverul le monitorizează.
10. **get_connections:** Afișează conexiunile de rețea active (established).
11. **get_memory:** Afișează informații detaliate despre memoria RAM și swap: total, disponibil, folosit, buffers, cached.
12. **get_sys_hist <count/data>:** Afișează istoricul utilizării memoriei și al procesorului. Argumentul `count` specifică numărul de intrări recente dorite, iar argumentul `data` poate fi o anumită zi sau un interval orar, împreună cu statistici min/max/avg.
13. **get_users_hist <count/data>:** Afișează frecvența conectării utilizatorilor, bazată pe ultimele `count` intrări din log.
14. **get_serv_hist <count/data> port:** Afișează istoricul activității conexiunilor per port (serviciu), cu statistici alăturate. Se poate filtra după un anumit port cu un al doilea argument opțional, "port".
15. **get_list_hist <count/data>:** Afișează istoricul porturilor deschise (listen) din loguri, pentru a vedea evoluția serviciilor oferite în timp.
16. **get_proc_hist <count/data>:** Afișează istoricul numărului de procese, al încărcării sistemului și al celor mai CPU-heavy procese înregistrate în loguri.
17. **set_interval <sec>:** Modifică intervalul de logare al serverului (între 1 și 3600 secunde). Folosește memorie partajată pentru sincronizare.
18. **get_interval:** Afișează intervalul de logare curent al serverului.

Tratarea Erorilor: Serverul validează fiecare comandă. Dacă o comandă este necunoscută, serverul răspunde cu mesajul "Comanda invalida.". Dacă utilizatorul nu este autentificat, răspunde cu "Eroare: Nu sunteți autentificat. Folosiți login.", menținând conexiunea activă pentru încercări ulterioare.

5 Scenarii de Utilizare

5.1 Scenarii de succes

1. **Verificare conexiuni:** Un administrator se autentifică (login admin admin) și verifică porturile deschise (get_services) și conexiunile externe suspicioase (get_connections) pentru a identifica potențiale infiltrări pe sistem.
2. **Acces concurent:** Doi administratori se conectează simultan. Unul dintre ei folosește get_load_avg, iar celalalt get_proc_info, pentru a identifica procesul care consumă resursele CPU. Serverul procesează cererile fără a bloca niciun utilizator.
3. **Monitorizare Vizuală (GUI):** Administratorul deschide clientul grafic și observă pe Dashboard un vârf în graficul "CPU Load". Printr-un simplu click pe tab-ul "Procese", identifică procesul vinovat evidențiat cu roșu (fiind rulat de root) și notează PID-ul.

5.2 Scenarii de eșec

1. **Utilizator neautentificat:** Administratorul încearcă să ruleze get_load_avg fără a fi logat. Serverul returnează "Eroare: Nu sunteți autentificat. Folositi login." și nu execută comanda.
2. **Fișier log inexistent:** Administratorul introduce o comandă de istoric (ex: get_memory_hist 10) imediat după pornirea serverului, înainte ca acesta să fi colectat date. Serverul returnează "Eroare: Nu exista loguri. Așteptati cateva minute pentru colectare." și menține conexiunea activă.
3. **Închidere forțată:** Administratorul închide brusc terminalul client fără a trimite comanda quit. Procesul copil al serverului detectează (read returnează 0) că socket-ul s-a închis și eliberează resursele asociate acelui client.

6 Concluzii

Soluția propusă implementează cu succes un sistem de monitorizare complet și eficient. Utilizarea selectului și fork() pentru izolarea clienților asigură scalabilitatea serverului pentru un număr rezonabil de clienți simultani (în jur de 1024), consumând resurse minime. Aplicația colectează periodic date din sistem (load average, memorie, utilizatori, conexiuni, servicii și procese) și le salvează în format CSV pentru analiză ulterioară. Memoria partajată permite modificarea dinamică a intervalului de logare din partea clienților, oferind flexibilitate în administrare. Autentificarea din fișier oferă un nivel basic de securitate.

7 Referinte Bibliografice

Bibliografie

1. Materiale laborator Rețele de Calculatoare. *TCP/IP server implementation, with I/O multiplexing using select()* <https://edu.info.uaic.ro/computer-networks/files/NetEx/S9/servTcpCSel.c>

2. Linux Man Pages. *select(2)*: <https://man7.org/linux/man-pages/man2/select.2.html>
3. Linux Man Pages. *proc(5)* <https://man7.org/linux/man-pages/man5/proc.5.html>
4. Linux Man Pages. *popen(3)* <https://man7.org/linux/man-pages/man3/popen.3.html>
5. Linux Man Pages. *errno(3)* <https://man7.org/linux/man-pages/man3/errno.3.html>
6. Stack Overflow. *Purpose of using fork() with select() in C sockets* <https://stackoverflow.com/questions/71914354/purpose-of-using-fork-with-select-in-c-sockets>
7. Linux Man Pages. *shm_open(3) - POSIX Shared Memory* https://man7.org/linux/man-pages/man3/shm_open.3.html
8. SFML (Simple and Fast Multimedia Library). *Official Documentation* <https://www.sfml-dev.org/documentation/2.6.1/>